

Engenharia de Processos de Software

Introdução à Estrutura da NBR ISO/IEC 12207

Prof. Ivan Granja

NBR ISO/IEC 12207 - Propósito

A norma internacional ISO/IEC 12207 tem como objetivo principal estabelecer uma estrutura comum (framework) para os **processos de ciclo de vida** e de **desenvolvimento de software**, visando ajudar as organizações a compreenderem todos os componentes que estão presentes na aquisição, desenvolvimento e fornecimento de software e, assim, conseguirem firmar contratos e executarem projetos de forma mais eficaz.

Para isso é necessário que compradores, fornecedores, desenvolvedores, mantenedores, operadores, gerentes e técnicos envolvidos no desenvolvimento e nas atividades inerentes ao ciclo de software usem uma linguagem (framework) comum.

NBR ISO/IEC 12207 – Terminologia Básica

O padrão ISO/IEC 12207:2008 define ao todo 25 processos, 95 atividades e 325 tarefas.

A Terminologia básica da norma segue uma hierarquia:

- **Tipos de Processos** (organizam e agrupam...)
 - **Processos** (são conjunto de ...)
 - **Atividades** (são constituídas por ...)
 - **Tarefas** (são executadas por pessoas)
 - **Produtos** (são os resultados das tarefas)

NBR ISO/IEC 12207 – Ciclo de Vida e Categorias (Tipos) de Processo

A ISO/IEC 12207:2008 divide os processos do **Ciclo de Vida** de Software em **quatro classes (categorias)**, segundo sua natureza

Ciclo de vida é o período de tempo que começa quando um produto de software é concebido e termina quando o software não está mais em uso. Tipicamente, inclui as fases de conceituação, requisitos, *design*, implementação, testes, instalação e verificação (implantação em produção) e por vezes uma fase de retirada de produção.

(Definição IEEE Std. 610.12-1990 - **IEEE Standard Glossary of Software Engineering Terminology.**)

NBR ISO/IEC 12207 – os 4 Tipos de Processos

1. Processos Fundamentais: contratação adquirente-fornecedor, execução do desenvolvimento, operação e manutenção do produto durante todo o ciclo de vida do produto.

2. Processos de Apoio: Auxiliam o trabalho e a qualidade do projeto de software.

3. Processos Organizacionais: Empregados por uma organização para estabelecer/implementar uma estrutura constituída pelos processos e pessoas envolvidas.

3.1 Processos de reuso, são uma subcategoria, realizados a fim de explorar sistematicamente oportunidades de reuso de software na organização

4. Processos de Adaptação: Atividades para adaptar a norma para especificidades da Organização ou de seu produto ou processo.

NBR ISO/IEC 12207 – Principais Processos por Categoria



ISO 12207 1 - Processos Fundamentais

1. **Processo de Aquisição:** atividades do adquirente (definir necessidades, emissão da proposta, seleção do fornecedor e da gerência do processo de aquisição).
2. **Processo de Fornecimento:** atividades do fornecedor (proposta para atendimento, contrato, definição de procedimentos e recursos – desde projeto, desenvolvimento, entrega e demais serviços contratados).
3. **Processo de Desenvolvimento:** atividades de análise de requisitos, projeto, codificação, integração, testes, instalação e aceitação. Divide-se em processo de operação e de Manutenção.

ISO 12207 1 - Processos Fundamentais

4. **Processo de Operação:** atividades do operador (organização que provê o serviço de operação do sistema no ambiente de funcionamento – envolve operação e suporte operacional aos usuários).
5. **Processo de Manutenção:** atividades do mantenedor (organização que provê os serviços de manutenção do software – modificações para mantê-lo adequado e atualizado).

ISO 12207 2 - Processos de Apoio

- 1 **Processo de Documentação**: atividades de planejar, produzir, editar, distribuir e manter atualizados os documentos necessários a todos os envolvidos.
- 2 **Processo de Gerência de Configuração**: atividades de identificar, definir e controlar os itens de um sistema (software, documentação, ferramentas etc.), controlando modificações, liberando versões e distribuição.
- 3 **Processo de Garantia da Qualidade**: atividades para garantir a conformidade dos processos e produtos de software em todo o ciclo de vida.

ISO 12207 2 - Processos de Apoio

- 4 **Processo de Verificação**: atividades para determinar se os produtos atendem completamente aos requisitos e condições impostas.
- 5 **Processo de Validação**: atividades para determinar se o produto final atende aos requisitos e ao uso específico proposto.
- 6 **Processo de Revisão Conjunta**: atividades para avaliar a situação e os produtos de uma atividade do projeto, tanto em níveis de gerenciamento do projeto como nos níveis técnicos do produto.

ISO 12207 2 - Processos de Apoio

7 Processo de Auditoria: atividades para determinar a adequação do produto aos requisitos, contrato, planos, padrões da instalação e outros que forem auditáveis.

8 Processo de Resolução de Problemas: atividades para analisar e resolver problemas (incluindo riscos e não-conformidade), detectados durante todo o ciclo de vida do produto. O objetivo é fornecer meios que garantam, em tempo adequado e de maneira responsável e técnica, a resolução de problemas e identificação de tendências de novas ocorrências.

ISO 12207 3 - Processos Organizacionais

1 Processo de Gerência: atividades genéricas empregadas por quaisquer das partes para gerenciar o processo.

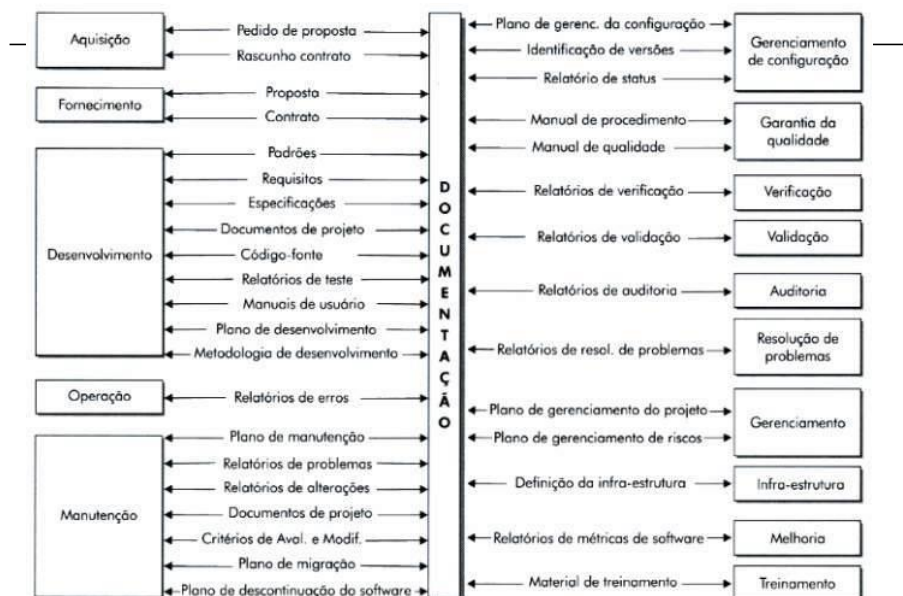
2 Processo de Infraestrutura: atividades para estabelecer e manter a infraestrutura necessária para os demais processos, como hardware, software, ferramentas, técnicas, padrões e recursos humanos necessários para o desenvolvimento, operação e manutenção do software.

ISO 12207 3 - Processos Organizacionais

3 Processo de Melhoria: atividades básicas para estabelecer, avaliar, medir, controlar e melhorar um processo do ciclo de vida.

4 Processo de Treinamento (Recursos Humano): atividades para oferecer e manter pessoal capacitado (planejar e implementar treinamentos com antecedência para poder disponibilizar esse recurso quando o produto de software for adquirido, desenvolvido, operado ou mantido).

Processos da ISO 12207 e os produtos relacionados



Para Refletir sobre o Rigor do processo

- ✓ “Alguns tem-se referido aos métodos prescritivos de processo (preditivos ou heurísticos) como “modelos de processo rigorosos”, por que eles frequentemente incluem as capacidades/competências sugeridas por normas como CMMi ou pelas ISO 12207 ou ISO 15504. No entanto, cada modelo de processo deve ser adaptado para que seja usado efetivamente em um projeto específico”.

Pressman, R.S. Engenharia de Software, 6ª Ed, p. 37.

Sete Princípios da Engenharia de Software

Primeiro princípio: *a razão de existir*

Um sistema de software existe por um motivo: *agregar valor para seus usuários*. Todas as decisões devem ser tomadas com esse princípio em mente. Antes de especificar um requisito de um sistema, antes de indicar alguma parte da funcionalidade de um sistema, antes de determinar as plataformas de hardware ou os processos de desenvolvimento, pergunte a si mesmo: “Isso realmente agrega valor real ao sistema?”. Se a resposta for “não”, não o faça. Todos os demais princípios se apoiam neste primeiro.

Segundo princípio: *KISS (Keep It Simple, Stupid!, ou seja: não complique!)*

O projeto de software não é um processo casual. Existem muitos fatores a considerar em qualquer trabalho de projeto. *Todo projeto deve ser o mais simples possível, mas não simplista*. Esse princípio contribui para um sistema mais fácil de compreender e manter. Isso não significa que características, até mesmo as internas, devam ser descartadas em nome da simplicidade. De fato, os projetos mais elegantes normalmente são os mais simples. Simples também não significa “gambiarra”. Na verdade, muitas vezes são necessárias muitas reflexões e trabalho em várias iterações para simplificar. A contrapartida é um software mais fácil de manter e menos propenso a erros.

Terceiro princípio: *mantenha a visão*

Uma visão clara é essencial para o sucesso. Sem ela, um projeto se torna ambíguo. Sem uma integridade conceitual, corre-se o risco de transformar o projeto em uma colcha de retalhos de projetos incompatíveis, unidos por parafusos inadequados... Comprometer a visão arquitetural de um sistema de software debilita e até poderá destruir sistemas bem projetados. Ter um arquiteto responsável e capaz de manter a visão clara e de reforçar a adequação ajuda a assegurar o êxito de um projeto.

Fonte: Engenharia de Software, Roger S. Presmann, 8ª Edição, pág 21 (em Português)

Sete Princípios da Engenharia de Software

Quarto princípio: *o que um produz outros consomem*

Raramente um sistema de software de qualidade industrial é construído e utilizado de forma isolada. De uma maneira ou de outra, alguém mais vai usar, manter, documentar ou, de alguma forma, depender da capacidade de entender seu sistema. Portanto, *sempre especifique, projete e implemente ciência de que mais alguém terá de entender o que você está fazendo*. O público para qualquer produto de desenvolvimento de software é potencialmente grande. Especifique tendo como objetivo os usuários. Projete tendo em mente os implementadores. Codifique se preocupando com aqueles que deverão manter e ampliar o sistema. Alguém terá de depurar o código que você escreveu, e isso o torna um usuário de seu código. Facilitando o trabalho de todas essas pessoas, você agrega maior valor ao sistema.

Quinto princípio: *esteja aberto para o futuro*

Um sistema com tempo de vida mais longo tem mais valor. Nos ambientes computacionais de hoje, em que as especificações mudam de um instante para outro, e as plataformas de hardware se tornam rapidamente obsoletas, a vida de um software, em geral, é medida em meses. Contudo, os verdadeiros sistemas de software com "qualidade industrial" devem durar muito mais. Para serem bem-sucedidos nisso, esses sistemas precisam estar prontos para se adaptar a essas e outras mudanças. Sistemas que obtêm sucesso são aqueles que foram projetados dessa forma desde seu princípio. *Jamais faça projetos limitados*. Sempre pergunte "e se" e prepare-se para todas as respostas possíveis, criando sistemas que resolvam o problema geral, não apenas o específico." Isso muito provavelmente conduziria à reutilização de um sistema inteiro.

Fonte: Engenharia de Software, Roger S. Presmann, 8ª Edição, pág 21 (em Português)

Sete Princípios da Engenharia de Software

Sexto princípio: *planeje com antecedência, visando a reutilização*

A reutilização economiza tempo e esforço.⁷ Alcançar um alto grau de reutilização é indiscutivelmente a meta mais difícil de ser atingida ao se desenvolver um sistema de software. A reutilização de código e projetos tem sido proclamada como uma grande vantagem do uso de tecnologias orientadas a objetos. Contudo, o retorno desse investimento não é automático. Aproveitar as possibilidades de reutilização – oferecidas pela programação orientada a objetos (ou convencional) – exige planejamento e capacidade de fazer previsões. Existem várias técnicas para levar a cabo a reutilização em cada um dos níveis do processo de desenvolvimento do sistema... *Planejar com antecedência para a reutilização reduz o custo e aumenta o valor tanto dos componentes reutilizáveis quanto dos sistemas aos quais eles serão incorporados*.

Sétimo princípio: *pense!*

Este último princípio é, provavelmente, o mais menosprezado. *Pensar bem e de forma clara antes de agir quase sempre produz melhores resultados*. Quando se analisa alguma coisa, provavelmente ela sairá correta. Ganha-se também conhecimento de como fazer correto novamente. Se você realmente analisar algo e mesmo assim o fizer da forma errada, isso se tornará uma valiosa experiência. Um efeito colateral da análise é aprender a reconhecer quando não se sabe algo, e até que ponto poderá buscar o conhecimento. Quando a análise clara faz parte de um sistema, seu valor aflora. Aplicar os seis primeiros princípios exige intensa reflexão, para a qual as recompensas em potencial são enormes.

Se todo engenheiro de software e toda a equipe de software simplesmente seguissem os sete princípios de Hooker, muitas das dificuldades enfrentadas no desenvolvimento de complexos sistemas baseados em computador seriam eliminadas.

Fonte: Engenharia de Software, Roger S. Presmann, 8ª Edição, pág 21 (em Português)